ChurnOps — Automated Churn Pipeline with Drift Alerts (PRD)

Oct 2, 2026 · @dharmi

1. Problem statement

ChurnOps is a GCP platform that keeps a churn model trustworthy on its own: it watches incoming subscriber data, detects drift, retrains, gates, and redeploys the model with no human in the loop, and uses an LLM to explain every incident in plain English.

Subscription businesses (OTT, music streaming, telecom) lose a share of subscribers every month. Most teams train a churn model once in a notebook and ship it. Then behaviour shifts: a price hike changes billing patterns, a new competitor cuts watch hours, a festival season spikes usage. The model silently degrades, and nobody notices until retention campaigns start missing.

The hard part is not the model. A gradient-boosted classifier on tabular data is a solved problem. The hard part is operations:

- **Detection** — knowing that the live data no longer looks like the training data, before labels arrive weeks later.
- **Reaction** — retraining, validating and redeploying safely, repeatably, without someone running a notebook.
- **Traceability** — knowing which data, code and config produced the model serving right now, and rolling back in one step.
- **Communication** — turning a wall of drift statistics into something a product manager can act on.

Framing for this project: this is a DevOps / platform project that happens to contain a model. Roughly 70% of the effort goes into infrastructure-as-code, pipelines, CI/CD/CT, event wiring, observability and the LLM operations layer; about 30% into data and modelling. The model is deliberately simple so the platform is the star.

2. Goals, non-goals and success metrics

Success means a live demo where we inject drift into the data and, with nobody touching anything, the platform detects it, retrains, gates, canary-deploys and posts an LLM-written incident report.

Goals

1. Fully automated loop: ingest → detect drift → retrain → evaluate → register → deploy, triggered by events, not people.
2. Everything as code: infrastructure (Terraform), pipelines (Kubeflow Pipelines on Vertex AI), deployments (Cloud Build), prompts (versioned YAML), dashboards.
3. Safe releases: quality gates, champion/challenger comparison, canary traffic split, one-command rollback.
4. Full lineage: every prediction traceable to model version → pipeline run → dataset snapshot → git commit.
5. LLMOps layer: an LLM "Drift Analyst" whose prompts, outputs, cost and quality are themselves versioned, evaluated and monitored.

Non-goals

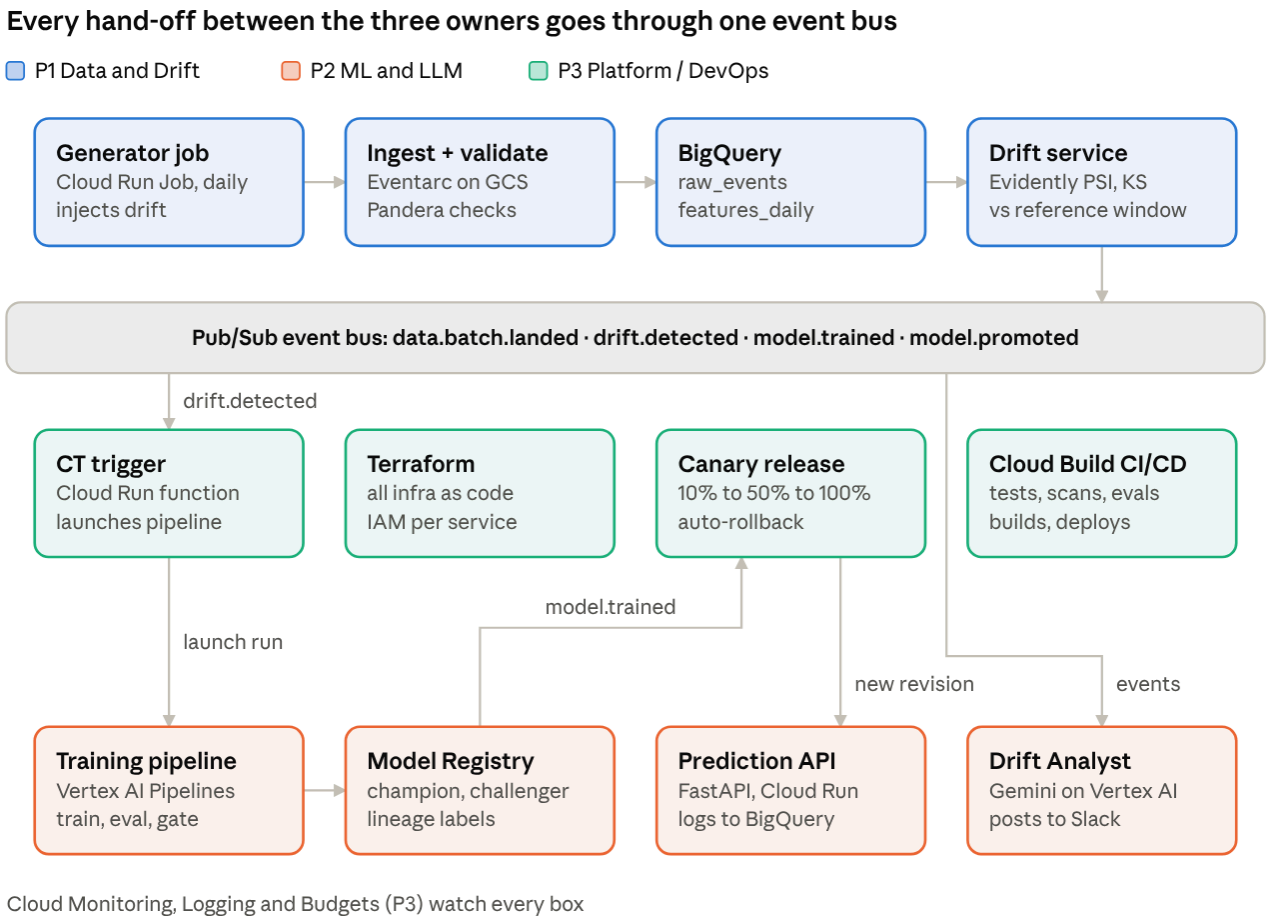
- State-of-the-art churn accuracy. A tuned XGBoost / LightGBM baseline is enough.
- Real customer data. We use a public dataset plus a synthetic generator.
- Multi-region, multi-cloud, or Kubernetes. Cloud Run + Vertex AI keeps it serverless and cheap.
- A full frontend product. Dashboards + Slack/email alerts are the UI.

Success metrics

Metric	Target
Drift injected → alert raised	under 15 min
Drift alert → new model serving (canary)	under 45 min, zero manual steps
Infra rebuilt from scratch with `terraform apply`	under 20 min, no console clicks
Bad model blocked by quality gate	100% in tests (we deliberately ship a broken one)
Rollback to previous champion	1 command / 1 pipeline, under 5 min
Prediction API p95 latency	under 300 ms
LLM report passes eval suite (faithfulness, actionability)	at least 85% of golden cases
Monthly GCP spend in dev	stays inside free-trial credits, budget alert at 50% / 90%

3. Solution overview and GCP architecture

ChurnOps is an event-driven, serverless system on GCP: small services that each do one job and talk only through Pub/Sub, BigQuery and the Vertex AI Model Registry.



ChurnOps on GCP · 12 components, 3 owners, 1 event bus

Read it top to bottom: P1's data plane emits `drift.detected`, P3's trigger launches P2's pipeline, P3's canary ships the model P2 registered, and P2's Drift Analyst narrates each step.

Layer	GCP services	Why this choice
Compute	Cloud Run services, Cloud Run Jobs, Cloud Run functions	Containers, scale to zero, cheap on credits, no cluster to babysit
Data	Cloud Storage, BigQuery	Landing zone + warehouse; BigQuery doubles as the log store for predictions and LLM calls
Events	Pub/Sub (with schemas + dead-letter topics), Eventarc, Cloud Scheduler	Loose coupling between the three owners

Layer	GCP services	Why this choice
ML	Vertex AI Pipelines, Training, Experiments, Model Registry, ML Metadata	Managed CT with lineage out of the box
LLM	Gemini on Vertex AI	Same project, same IAM, no external keys
Delivery	Cloud Build, Artifact Registry, Terraform, Workload Identity Federation	Keyless, reproducible, all in git
Ops	Cloud Logging, Monitoring, Trace, Error Reporting, Budgets, Secret Manager	One place to see health, cost and incidents

4. MLOps and LLMOps design

We aim for Google's MLOps "level 2": CI/CD for code plus CT (continuous training) for the model, with the LLM layer held to the same release discipline.

4.1 The three loops

Loop	Trigger	What runs	Owner
CI — continuous integration	Pull request	Lint, unit tests, contract tests, Terraform plan, container build, prompt eval suite	Person 3 (pipeline), everyone (tests)
CD — continuous delivery	Merge to `main`	Push images to Artifact Registry, `terraform apply` to dev, compile + upload pipeline spec, deploy services	Person 3
CT — continuous training	`drift.detected` event or weekly schedule	Vertex AI Pipeline: snapshot → validate → train → evaluate → gate → register → canary	Person 2 (pipeline), Person 3 (deploy)

4.2 MLOps concepts we implement

- **Data validation** — schema + range checks on every batch before it touches BigQuery tables; bad batches quarantined in GCS.
- **Drift detection** — feature drift (PSI, KS test, Jensen-Shannon via Evidently) and prediction drift on the model's output distribution, against a frozen reference window.
- **Experiment tracking** — Vertex AI Experiments logs params, metrics, dataset version per training run.
- **Model registry** — Vertex AI Model Registry with aliases `champion` and `challenger`; labels carry git SHA, data snapshot ID, pipeline run ID.
- **Quality gates** — challenger must beat champion on the fresh holdout (ROC-AUC, recall at fixed precision) and pass fairness/sanity checks, or the pipeline stops.
- **Progressive delivery** — Cloud Run revision traffic split 10% → 50% → 100%, auto-rollback if error rate or latency SLO breaches.
- **Lineage and reproducibility** — pinned container digests, dataset snapshots in BigQuery, pipeline artifacts in Vertex ML Metadata.
- **Monitoring** — Cloud Monitoring SLOs for the API, plus BigQuery prediction logs for model-level metrics.

4.3 LLMOps: the Drift Analyst

The LLM component is a **Drift Analyst** built on a Gemini model in Vertex AI. It subscribes to `drift.detected` and `model.promoted` events, reads the drift report and pipeline results, and writes an incident summary: what drifted, the likely business cause, whether retraining helped, and what to check next. It posts to Slack/email and stores the report in BigQuery. A stretch goal is a "chat with your pipeline" endpoint that answers questions like "why did v7 replace v6?" using the stored reports as context (RAG over BigQuery).

The LLM is treated as a deployable artifact, not a magic API call:

- **Prompt versioning** — prompts live as YAML in git (`llm/prompts/drift_analyst/v3.yaml`) with model name, temperature, and output schema. The active version is a config value, so a prompt change is a PR.
- **Structured output** — the response must validate against a JSON schema (Pydantic); invalid output is retried once, then falls back to a templated summary.
- **Offline evaluation** — a golden set of ~30 drift scenarios with expected key findings. CI runs every prompt change against it, scoring faithfulness (no invented numbers), coverage of top drifted features, and actionability, using rule checks plus an LLM-as-judge.
- **Guardrails** — numbers in the summary are cross-checked against the drift report; PII-like fields never enter the prompt.

- **Observability** — every call logs prompt version, model, latency, input/output tokens, estimated cost and eval score to BigQuery; Cloud Monitoring alerts on latency, error rate and daily token spend.
- **Release flow** — a new prompt version runs in shadow mode on real events before it becomes active.

5. Data, contracts and repo structure

The three of us can work in parallel from day one only because every hand-off is a written contract, checked into `contracts/` before anyone builds against it.

5.1 Data source

- **Seed data:** IBM Telco Customer Churn (Kaggle, ~7,000 customers, tenure, contract type, monthly charges, services, churn label). Optionally the KKBox churn dataset later if we want streaming-hours style features.
- **Synthetic generator:** a Cloud Run Job that samples from the seed distribution and emits a daily batch of ~2,000 customer-activity rows, with a `drift_scenario` config so we can inject realistic shifts on demand: `price_hike` (MonthlyCharges +25%), `competitor_launch` (tenure and usage drop for one segment), `new_plan` (unseen category value), `seasonal_spike`.
- **Delayed labels:** churn labels for a batch arrive 2 simulated weeks later, which is exactly why we need drift as an early-warning signal.

5.2 Contracts

Contract	Lives in	Owner	Consumers
BigQuery table schemas: `raw_events`, `features_daily`, `reference_window`, `predictions_log`, `drift_reports`, `llm_calls`	`contracts/bq/*.json`	P1 (data tables), P2 (`predictions_log`, `llm_calls`)	All
Pub/Sub topics + Avro/JSON schemas: `data.batch.landed`, `drift.detected`, `model.trained`, `model.promoted`, `model.rejected`	`contracts/events/*.json`	P3 (topics), producer team (payload)	All

Contract	Lives in	Owner	Consumers
Feature list + transform spec	`contracts/features.yaml`	P1	P2 (training), P2 (serving)
Prediction API (OpenAPI): `POST /predict`, `GET /health`, `GET /version`	`contracts/api/openapi.yaml`	P2	P1 (batch scoring), P3 (smoke tests)
GCS layout: `gs://<proj>-landing/`, `-quarantine/`, `-artifacts/`, `-reports/`	`contracts/storage.md`	P3	All
Model registry labels + aliases	`contracts/registry.md`	P2	P3 (deploy), P2 (LLM)

Every event payload carries a `trace_id`, `batch_id`, `schema_version` and `emitted_at`, so one ID follows a batch through every service and into the LLM report.

5.3 Monorepo layout

```
churnops/
  contracts/        # schemas, OpenAPI, event specs (reviewed by all 3)
  infra/            # Terraform: modules/ + envs/dev, envs/prod  (P3)
  data/
    generator/      # synthetic batch + drift injection   (P1)
    ingest/         # validation + load to BigQuery        (P1)
    drift/          # Evidently drift job                  (P1)
  ml/
    pipeline/       # KFP components + pipeline definition (P2)
    serving/        # FastAPI prediction service           (P2)
  llm/
    analyst/        # Drift Analyst service                (P2)
    prompts/        # versioned prompt YAMLs               (P2)
    evals/          # golden set + eval runner             (P2)
  ops/
    trigger_fn/     # event -> pipeline launcher           (P3)
    dashboards/     # Monitoring JSON, Looker Studio       (P3)
  cloudbuild/       # CI/CD configs                        (P3)
  tests/integration # end-to-end tests                     (all)
```

Branching: trunk-based, short feature branches, PRs need one reviewer from another person and a green CI run.

6. Team split

The work splits along the event flow: Person 1 owns everything up to `drift.detected`, Person 2 owns everything from `drift.detected` to a registered model plus the LLM layer, and Person 3 owns the platform all of it runs on and the path to production.

Role	Owns	Key GCP services	Emits	Consumes
P1 — Data & Drift Engineer	Generator, ingestion, validation, feature tables, drift detection, batch scoring	Cloud Storage, BigQuery, Cloud Run Jobs, Cloud Scheduler, Eventarc, Pub/Sub	`data.batch.landed`, `drift.detected`	Prediction API, `model.promoted` (to refresh reference window)
P2 — ML & LLM Engineer	Training pipeline, evaluation gates, model registry, prediction service, Drift Analyst, prompt evals	Vertex AI Pipelines, Training, Experiments, Model Registry, Gemini on Vertex AI, Cloud Run	`model.trained`, `model.rejected`, `model.promoted`	`drift.detected`, feature tables
P3 — Platform / DevOps Engineer	Terraform, IAM, CI/CD/CT, Artifact Registry, pipeline trigger, canary + rollback, observability, cost	Terraform, Cloud Build, Artifact Registry, Workload Identity Federation, Secret Manager, Cloud Monitoring/Logging, Budgets	Deploy events, alerts	Every image, every event, every log

Everyone writes their own Dockerfile, unit tests and the Terraform variables their service needs; P3 reviews and owns the modules. Rotate PR reviews so each person reads the other two codebases — that is what makes the integration days smooth and what lets each of you explain the whole system in an interview.

7. Person 1 — Data & Drift Engineer

P1 delivers a self-running data plane: every day a batch lands, is validated, turned into features, scored, and compared against the reference window, ending in either silence or a `drift.detected` event.

Phase A — Foundations (week 1)

1. EDA on the Telco dataset in a notebook: distributions, missing values (`TotalCharges` blanks), class balance (~26% churn). Save plots to `docs/eda/`.
2. Write `contracts/features.yaml`: every feature name, dtype, allowed range or categories, and transform (one-hot, log, bucket). Agree it with P2 before week 1 ends — this is the single most important hand-off.
3. Write `contracts/bq/raw_events.json`, `features_daily.json`, `reference_window.json`, `drift_reports.json` (BigQuery JSON schema format), partitioned by `batch_date`, clustered by `customer_segment`.
4. Upload the cleaned seed dataset to `gs://<proj>-landing/seed/` and load it as `raw_events` partition zero.

Phase B — Synthetic generator with drift injection (week 2)

1. `data/generator/`: Python package that fits per-column distributions (and simple correlations, e.g. tenure vs contract) from the seed data, then samples N rows per run.
2. Add `scenarios.yaml` with named drift scenarios and a severity knob (0–1). Each scenario is a function that transforms the sampled frame (shift a mean, change a category mix, add an unseen category).
3. Generate delayed labels: write labels for batch *d* only when batch *d+14* is generated, into a `labels` table.
4. Containerise (slim Python base, non-root user), run as a **Cloud Run Job** with args `--scenario --severity --date`.
5. P3 wires **Cloud Scheduler** to run it daily; you also expose a manual trigger (a `make inject-drift SCENARIO=price_hike` target) for the demo.
6. Output: a Parquet file in `gs://<proj>-landing/batches/date=YYYY-MM-DD/`.

Phase C — Ingestion and validation (week 2–3)

1. `data/ingest/`: a Cloud Run Job triggered by **Eventarc** on object-finalize in the landing bucket.
2. Validate with **Pandera** (or Great Expectations): schema, nulls, ranges, category sets from `features.yaml`. Failing batch → moved to `-quarantine/`, structured error logged, alert raised; nothing reaches BigQuery.
3. Passing batch → load to `raw_events` (idempotent: `MERGE` on `batch_id` so reruns never duplicate).
4. Build `features_daily` with a SQL transform (or Dataform) that implements exactly `features.yaml` — the same logic P2's serving uses (share one `features` Python module, or compute features only in BigQuery and serve from there).

5. Publish `data.batch.landed` to Pub/Sub with `batch_id`, row count, `trace_id`.

Phase D — Drift detection (week 3–4)

1. `data/drift/`: a Cloud Run service subscribed (push subscription) to `data.batch.landed`.
2. Load the current batch and the `reference_window` (the data the current champion was trained on — refreshed when `model.promoted` fires).
3. Run **Evidently** data-drift preset: per-feature PSI for numeric, chi-square / Jensen-Shannon for categorical, plus share of drifted features. Also prediction drift: call P2's `/predict` in batch (or read `predictions_log`) and compare score distributions.
4. Decision rule in `drift_policy.yaml` (config, not code): e.g. drift if PSI > 0.2 on any top-5 importance feature, or more than 30% of features drifted, or prediction drift p < 0.01. Severity levels `none / warn / critical`.
5. Write the full report JSON + HTML to `gs://<proj>-reports/drift/<batch_id>/` and a summary row to `drift_reports`.
6. If `critical` → publish `drift.detected` (payload: batch_id, drifted features with scores, severity, report URI, trace_id). Add a **cooldown** (no second trigger within N hours) so one drifted week doesn't launch ten trainings.
7. When labels arrive, compute realised performance (actual AUC of the champion on labelled batches) and write it to `drift_reports` — this proves drift alerts actually predicted degradation.

Phase E — Hardening and tests (week 4–5)

1. Unit tests for every scenario (injecting `price_hike` must move MonthlyCharges PSI above threshold; injecting nothing must not).
2. Contract tests: generated batch validates against `features.yaml`; emitted events validate against `contracts/events/*.json`.
3. Data-quality dashboard panel spec for P3: rows per batch, quarantine count, PSI per feature over time.
4. Write the runbook section "what to do when a batch is quarantined".

Definition of done: `make inject-drift SCENARIO=price_hike` produces a `drift.detected` event within 15 minutes, with a report that names MonthlyCharges as the top drifted feature.

8. Person 2 — ML & LLM Engineer

P2 delivers the continuous-training pipeline that turns a `drift.detected` event into a gated, registered challenger model, the prediction service that serves it, and the LLM Drift Analyst with its full LLMOps loop.

Phase A — Baseline model (week 1)

1. With P1's `features.yaml`, train a baseline locally: logistic regression and XGBoost/LightGBM, stratified split, class weights for imbalance.
2. Pick metrics: ROC-AUC (primary), recall at 70% precision (business metric: how many churners a retention campaign catches), calibration (Brier score).
3. Package training as a script `ml/pipeline/train.py` that takes a BigQuery table + snapshot date and writes a model artifact (`model.joblib` + `metadata.json`) to GCS. No notebooks in the pipeline.
4. Write `contracts/api/openapi.yaml` and `contracts/registry.md` (aliases `champion` / `challenger`; labels `git_sha`, `data_snapshot`, `pipeline_run`, `drift_trace_id`).

Phase B — Vertex AI training pipeline (week 2–3)

Build with the **Kubeflow Pipelines SDK (KFP v2)**, one container per component, compiled to YAML and run on **Vertex AI Pipelines**:

1. `snapshot_data` — creates an immutable BigQuery snapshot table of `features_daily` + `labels` up to the drift date. Output: snapshot ID.
2. `validate_data` — re-checks schema and label balance; fails fast.
3. `split` — time-based split (train on older, evaluate on most recent labelled weeks), so evaluation reflects the drifted world.
4. `train` — runs as a Vertex AI custom training job; small hyperparameter sweep (or Vertex Vizier, stretch). Logs to **Vertex AI Experiments**.
5. `evaluate` — scores challenger *and* current champion on the same fresh holdout; writes a metrics JSON + confusion matrix.
6. `gate` — a `dsl.If` condition: challenger AUC ≥ champion AUC − 0.005 *and* recall@70P improves *and* sanity checks pass (no feature with zero importance that used to dominate, prediction rate within bounds). Thresholds live in `gate_policy.yaml`.
7. `register` — if gate passes, upload to **Vertex AI Model Registry** as a new version with alias `challenger` and all lineage labels; publish `model.trained`. If gate fails, publish `model.rejected` with reasons.
8. Pipeline parameters: `drift_trace_id`, `snapshot_date`, `trigger_reason` (drift / schedule / manual).

Phase C — Prediction service (week 3)

1. `ml/serving/`: **FastAPI** app with `/predict` (single + batch), `/health`, `/version`.
2. On startup it reads the registry alias it is told to serve (env var `MODEL_ALIAS`), downloads that artifact, and reports its version in `/version` and in every response.
3. Pydantic request models generated from `features.yaml`; reject out-of-schema input with 422.
4. Log every prediction (request ID, model version, features hash, score, latency) to `predictions_log` — async via Pub/Sub → BigQuery subscription so it never slows the request.
5. Deployed on **Cloud Run** by P3's pipeline (scales to zero, cheap). Vertex AI Endpoint is a stretch option only, because it bills while idle.
6. Load test with Locust: confirm p95 under 300 ms at 20 RPS.

Phase D — Drift Analyst (LLM) (week 3–4)

1. `llm/analyst/`: Cloud Run service, push-subscribed to `drift.detected`, `model.rejected` and `model.promoted`.
2. Context builder: pulls the drift summary (top drifted features + scores), the gate metrics, and the last 3 incidents from BigQuery. Keep the prompt small and numeric; never send raw customer rows.
3. Call a Gemini Flash-tier model on Vertex AI (pick the current GA version) with the active prompt from `llm/prompts/drift_analyst/`. Force JSON output: `headline`, `what_changed[]`, `likely_causes[]`, `model_action`, `recommended_checks[]`, `confidence`.
4. Guardrail: every number in the output must appear in the input context (regex + compare); otherwise retry, then fall back to a templated report.
5. Deliver: Slack webhook (secret in Secret Manager) or email, plus a row in `incident_reports`.
6. Log each call to `llm_calls`: prompt version, model, tokens in/out, latency, cost estimate, guardrail result.

Phase E — LLMOps evaluation loop (week 4–5)

1. Build `llm/evals/golden/`: ~30 drift scenarios (generate them by running P1's generator with each scenario and severity), each with the expected key findings written by you.
2. Eval runner scores each prompt version on: faithfulness (no invented numbers — rule check), coverage (mentions the top-3 drifted features), actionability and clarity (LLM-as-judge with a rubric, or Vertex AI's Gen AI evaluation service).

3. Wire it into CI with P3: any PR touching `llm/prompts/` runs the eval suite and posts the score table on the PR; merge blocked if score drops below the current version.
4. Shadow mode: a new prompt version runs on live events alongside the active one for a few days; compare in a BigQuery view before switching `ACTIVE_PROMPT_VERSION`.
5. Stretch: "Ask ChurnOps" endpoint — a question goes in, relevant incident reports and pipeline runs are retrieved from BigQuery (vector search or plain SQL filters), and Gemini answers with citations to run IDs.

Definition of done: a `drift.detected` event produces either a registered challenger or a `model.rejected` event within 30 minutes, and a Slack message from the Drift Analyst that passes the guardrail.

9. Person 3 — Platform / DevOps Engineer

P3 delivers the platform: a GCP project that can be rebuilt from git in one command, a CI/CD/CT system that moves code and models to production safely, and the observability that proves it all works.

Phase A — Project bootstrap and IaC (week 1)

1. Create the GCP project, enable billing (free-trial credits), and immediately set a **Budget** with alerts at 50% and 90%.
2. Bootstrap Terraform remote state in a versioned GCS bucket (the one resource created by hand or a tiny bootstrap script).
3. Write Terraform modules in `infra/modules/`: `apis` (enable services), `storage` (landing, quarantine, artifacts, reports buckets with lifecycle rules), `bigquery` (dataset + tables from `contracts/bq/*.json`), `pubsub` (topics + schemas from `contracts/events/` + dead-letter topics), `artifact_registry`, `cloud_run_service`, `cloud_run_job`, `scheduler`, `eventarc`, `secrets`, `monitoring`.
4. IAM: one service account per component (`sa-generator`, `sa-ingest`, `sa-drift`, `sa-pipeline`, `sa-serving`, `sa-analyst`, `sa-cicd`) with least privilege. No JSON keys anywhere.
5. `infra/envs/dev` composes the modules. Goal for end of week 1: `terraform apply` creates every bucket, table and topic the others need, so they build against real infra from week 2.

Phase B — CI/CD (week 2)

1. Connect GitHub to **Cloud Build** (or GitHub Actions authenticating via **Workload Identity Federation** — keyless).

2. **CI on every PR** (`cloudbuild/ci.yaml`): ruff + mypy, unit tests per package, contract tests (schemas validate, OpenAPI unchanged or versioned), `terraform fmt` + `validate` + `plan` (plan posted as PR comment), container builds, Trivy/Artifact Analysis vulnerability scan, and, for `llm/prompts/**` changes, P2's eval suite.
3. **CD on merge to main** (`cloudbuild/cd.yaml`): build and push images tagged with git SHA to Artifact Registry, `terraform apply` in dev, deploy Cloud Run services/jobs by **image digest**, compile the KFP pipeline and upload it to an Artifact Registry KFP repo (versioned templates).
4. Path filters so a change in `data/` doesn't redeploy the analyst.
5. Post-deploy smoke tests: `/health` and `/version` on serving, a test message through each Pub/Sub topic.

Phase C — Continuous training trigger and model release (week 3)

1. `ops/trigger_fn/`: a Cloud Run function subscribed to `drift.detected` that launches the latest pipeline template on Vertex AI Pipelines with `drift_trace_id` and `snapshot_date` — idempotent on `trace_id`, and respecting a concurrency lock (one CT run at a time).
2. Also a weekly Cloud Scheduler job that launches the same pipeline with `trigger_reason=schedule`.
3. Release pipeline (`cloudbuild/release-model.yaml`), triggered by `model.trained`:
   a. Deploy a new Cloud Run revision of the serving service pointing at the `challenger` version, tagged `canary`, with 0% traffic.
   b. Smoke-test the tagged URL (golden requests, schema, latency).
   c. Shift 10% traffic; wait and watch error rate + p95 latency in Cloud Monitoring.
   d. If SLOs hold, 50% then 100%; move registry alias `champion` to the new version, publish `model.promoted`.
   e. If anything breaches, route 100% back to the previous revision, publish `model.rejected` with reason `canary_failed`.
4. `make rollback` / a manual Cloud Build trigger: point traffic and the `champion` alias back to the previous version in one step.

Phase D — Observability and SRE (week 4)

1. Structured JSON logging convention for every service (`trace_id`, `batch_id`, `model_version`, `severity`) — publish it in `contracts/logging.md` week 1.
2. **Cloud Monitoring dashboards as code** (`ops/dashboards/*.json`, applied by Terraform): Platform (request rate, errors, p95, instance count), Data (batches, quarantines, PSI per feature from log-based metrics), Model (champion version, AUC

over time, CT runs pass/fail), LLM (calls, tokens, cost/day, guardrail failures).
3. **SLOs + alert policies**: serving availability 99%, p95 < 300 ms; pipeline failure; dead-letter messages > 0; daily LLM spend over budget.
4. A Looker Studio report on BigQuery for the demo: drift timeline with incidents, model versions and LLM headlines on one page.
5. Cloud Trace on the services so one `trace_id` can be followed across the flow.

Phase E — Security, cost and chaos (week 5)

1. Secret Manager for the Slack webhook and any keys; no secrets in env files or git (add `gitleaks` to CI).
2. Cloud Run min-instances 0, Artifact Registry cleanup policies, GCS lifecycle deletes after 30 days, BigQuery partition expiry on logs.
3. Chaos tests before demo day: push a deliberately broken model (gate must block it), kill the serving revision mid-canary (rollback must fire), publish a malformed event (dead-letter must catch it).
4. `make destroy` + `make up` dry run: rebuild the whole environment from zero and time it.

Definition of done: a fresh clone + `make up` produces a working platform; a merge to main deploys with no manual step; a `model.trained` event ends in either a promoted model or an automatic rollback.

10. Connecting the three parts end to end

The three parts never call each other's internals: they connect only through Pub/Sub events, BigQuery tables, the registry, and one HTTP API, all defined in `contracts/` and all provisioned by P3's Terraform.

10.1 The end-to-end flow (one `trace_id` all the way)

1. **Cloud Scheduler** (P3) runs the generator job (P1) → Parquet batch in `-landing/`.
2. **Eventarc** (P3) fires the ingest job (P1) → validate → load `raw_events` → build `features_daily` → publish **`data.batch.landed`**.
3. Drift service (P1) receives it → calls `/predict` on serving (P2) for prediction drift → writes `drift_reports` → if critical, publish **`drift.detected`**.
4. Two subscribers react in parallel:
   a. Trigger function (P3) launches the Vertex AI Pipeline (P2).
   b. Drift Analyst (P2) sends the first incident message: "drift detected, retraining started".

5. Pipeline (P2) snapshots, trains, evaluates, gates → publishes **`model.trained`** (or **`model.rejected`**).
6. Release pipeline (P3) canaries the challenger on Cloud Run → promotes → moves `champion` alias → publishes **`model.promoted`** (or rolls back).
7. On `model.promoted`: drift service (P1) refreshes `reference_window` to the new training snapshot; Drift Analyst (P2) posts the closing report ("v8 replaced v7, AUC 0.81 → 0.84 on post-drift data").
8. Dashboards (P3) show the whole incident on one timeline, joined on `trace_id`.

10.2 How we integrate without blocking each other

Hand-off	Real producer ready	Mock until then
Buckets, tables, topics, service accounts	P3, end of week 1	none — this is the first deliverable
`features.yaml` + BigQuery schemas	P1, end of week 1	none — agreed in a meeting
`data.batch.landed` / `drift.detected`	P1, week 3–4	`scripts/publish_fake_event.py` with sample payloads from `contracts/events/examples/`
A model in the registry	P2, week 3	dummy logistic-regression model registered in week 1
Prediction API	P2, week 3	stub FastAPI returning 0.5, same OpenAPI
Pipeline trigger + release	P3, week 3	P2 launches pipeline manually from CLI

10.3 Integration rituals

- **Week 1 walking skeleton:** every box exists and is wired, even if it does nothing — generator writes 10 rows, ingest loads them, drift always says "critical", pipeline registers the dummy model, canary deploys it, analyst posts "hello". Ugly but end to end.
- **Integration Friday:** each week, merge everything to `main`, run `tests/integration/test_e2e.py` (injects a scenario, polls BigQuery and the registry until `model.promoted` or timeout), fix what broke together.
- **Contract changes** need a PR approved by both producer and consumer, and bump `schema_version`.

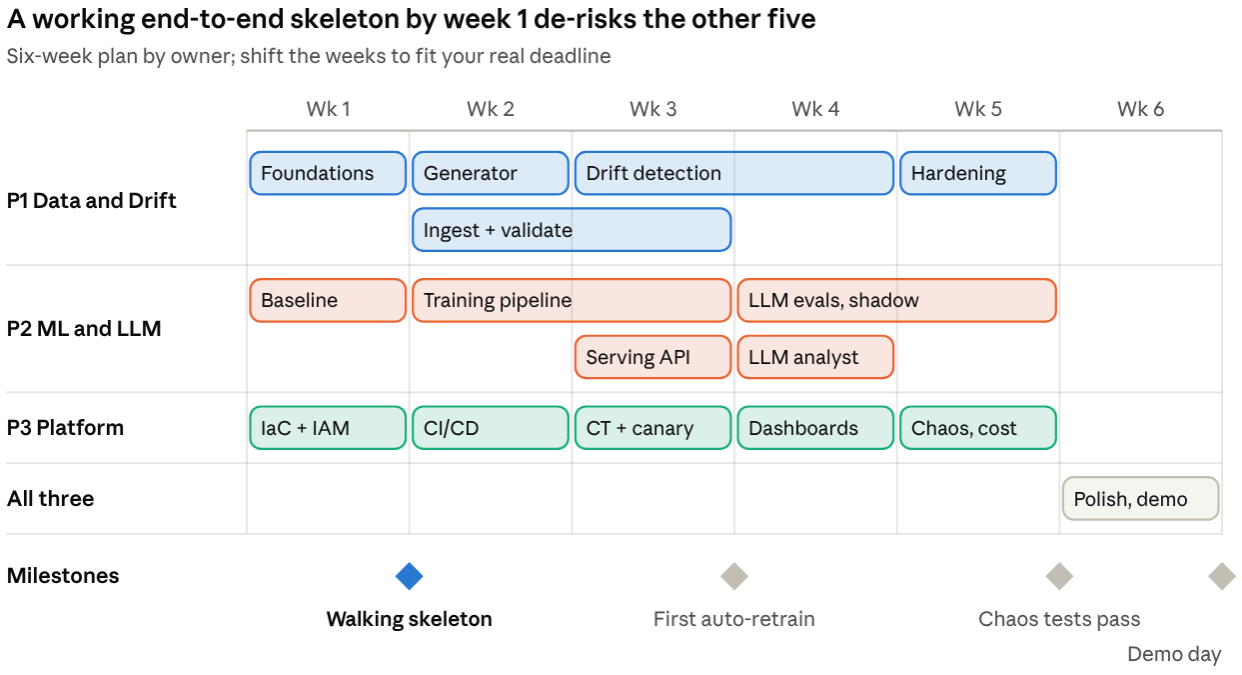
- **Shared debugging:** filter Cloud Logging by `trace_id` to see one incident across all services.

10.4 Demo script (5 minutes)

1. Show the dashboard: champion v3, healthy, no drift.
2. Run `make inject-drift SCENARIO=price_hike SEVERITY=0.8`.
3. Within minutes: drift alert on the dashboard, Slack message from the Drift Analyst explaining MonthlyCharges shift.
4. Show the Vertex AI Pipeline running, then the gate passing.
5. Show Cloud Run traffic moving 10% → 100%, `champion` alias moving to v4.
6. Closing LLM report with before/after metrics; then inject a broken model to show the gate blocking it.

11. Timeline

The plan assumes six weeks; the one date that must not slip is the week-1 walking skeleton, because every later week builds on it.



6-week plan · 3 owners, 4 milestones

Every Friday is an integration day: merge to `main`, run the end-to-end test, fix together. If you have fewer weeks, compress phases but keep the skeleton first and the demo rehearsal last.

12. Risks, cost control and open questions

The biggest risk is not technical: it is three people building in isolation and discovering in week 5 that their pieces don't fit. The walking skeleton in week 1 exists to kill that risk.

Risk	Impact	Mitigation
Integration surprises late	Demo breaks	Contracts first, walking skeleton by end of week 1, integration day every Friday
GCP bill overrun	Credits gone mid-project	Budget alerts, Cloud Run scale-to-zero, no idle Vertex endpoints, small machine types, lifecycle rules, `make destroy` after demos
Vertex AI Pipelines slow to iterate	Lost days	Run components locally with KFP local runner / plain Python first; pipeline only wraps tested code
Synthetic drift looks fake	Weak story	Base scenarios on real events (price hike, competitor launch) and tie each to a business narrative in the LLM report
Drift alert storms	Many redundant trainings	Cooldown window, concurrency lock, severity levels
LLM hallucinates numbers	Misleading incident report	Numeric guardrail, JSON schema, templated fallback, eval suite in CI
One person blocked on another	Idle time	Mocks: P1 publishes fake `drift.detected` early; P2 ships a dummy model to the registry in week 1; P3 provides infra by week 1
Service-account permission errors	Hours lost to 403s	IAM fully in Terraform, documented per service, tested in the skeleton

Open questions

- [ ] How long do we actually have until the deadline / demo? (The timeline assumes 6 weeks.)
- [ ] Cloud Build or GitHub Actions + Workload Identity Federation for CI?
- [ ] Slack or email as the alert channel for the demo?
- [ ] Do we need a prod environment, or is dev + a demo-day freeze enough?

- [ ] Is the "Ask ChurnOps" RAG endpoint in scope or stretch?